학습 교안

하네스 엔지니어링 입문 1단계

AI 에이전트에게 맡기되, 규칙과 검증으로 통제하는 생초보를 위한 첫걸음

OpenCode · OpenRouter · Python · pytest · Git

0장 준비 + 1~6장 · Docker 없이 진행하는 입문 실습

강사 김대철

작성일 2026.11.13

실습 환경 Windows + Python + Git + OpenCode

실습 저장소 https://github.com/sinaboro/harness-beginner-tutorial

교안의 명령과 테스트 결과는 Windows PowerShell 에서 실제로 실행해 얻은 것입니다.
AI 의 답변은 실행할 때마다 달라지므로, 교안에는 답변 대신 확인 방법을 실었습니다.

목차

과정 안내 5

단계 안내 5

2단계 교시표 5

도커로 실행하기 (선택) 5

0단계 교시표 6

1단계 교시표 6

저장소 구조 7

시작하기 7

원칙 세 가지 7

강사 안내 8

1단계 준비하기 (도커) 9

1. Docker Desktop 설치 (Windows) 9

2. 저장소 받기와 설정 파일 (Windows) 9

3. 작업 환경 만들기 (Windows, 처음 한 번) 9

4. 컨테이너 들어가기 (Windows) 10

5. OpenRouter 연결 (컨테이너 안) 10

6. 작업 폴더 확인 10

확인 체크리스트 10

자주 만나는 오류 10

1단계 1장. 하네스란 무엇인가: 같은 부탁, 다른 결과 12

이 장에서 할 일 한눈에 보기 12

1단계. 실험 A: 조건 없이 한 줄로 시키기 (15분) 12

2단계. 실험 A 결과 확인하고 표 A 채우기 (25분) 12

3단계. 실험 B: 조건 세 개를 붙여 시키기 (25분) 13

4단계. 표 B 채우고 표 A 와 비교하기 (20분) 13

5단계. 조건 세 개가 막으려는 것 적기 (15분) 14

확인 체크리스트 14

자주 만나는 오류 14

1단계 2장. 요구사항 쓰기: 계산기 요구사항 한 장 15

이 장에서 할 일 한눈에 보기 15

1단계. 흐린 말 고치기 (15분) 15

2단계. 요구사항 파일 만들기 (5분) 15

3단계. 워크시트대로 칸 채우기 (40분) 16

4단계. 점검 항목 확인하고 고치기 (25분) 16

5단계. 옆 사람과 바꿔 읽고 질문 적기 (15분) 17

확인 체크리스트 . . . 17
자주 만나는 오류 . . . 17
1단계 3장. 작업 지시서 쓰기 . . . 18
이 장에서 할 일 한눈에 보기 . . . 18
1단계. 지시서 파일 만들기 (5분) . . . 18
2단계. 여섯 칸 워크시트 채우기 (35분) . . . 18
3단계. AI 에게 지시서 건네기 (15분) . . . 19
4단계. 계획 검토하고 승인하기 (20분) . . . 19
5단계. 결과를 지시서와 대조하기 (20분) . . . 20
6단계. 1장 실험 A 와 비교하기 (15분) . . . 20
확인 체크리스트 . . . 20
자주 만나는 오류 . . . 21
1단계 4장. 규칙과 권한 세우기 . . . 22
이 장에서 할 일 한눈에 보기 . . . 22
1단계. 작업 폴더 만들기 (10분) . . . 22
2단계. AGENTS.md 의 [작성] 17개 채우기 (35분) . . . 23
3단계. 권한 설정 읽고 답하기 (15분) . . . 24
4단계. 막혀야 할 일을 AI 에게 시키고 결과 적기 (25분) . . . 24
5단계. AI 가 규칙을 읽었는지 확인하기 (10분) . . . 25
확인 체크리스트 . . . 25
자주 만나는 오류 . . . 25
1단계 5장. 검증하고 저장하기 . . . 26
이 장에서 할 일 한눈에 보기 . . . 26
1단계. 정리하고 빨강 확인하기 (15분) . . . 26
2단계. AI 에게 multiply 추가시키기 (30분) . . . 27
3단계. 초록 확인하고 테스트가 바뀌지 않았는지 대조하기 (15분) . . . 27
4단계. 커밋 전에 검증하는 장치 켜기 (10분) . . . 28
5단계. 일부러 틀리게 고쳐서 저장이 막히는지 보기 (20분) . . . 28
6단계. 정식으로 저장하기 (20분) . . . 28
확인 체크리스트 . . . 29
자주 만나는 오류 . . . 29
1단계 6장. 미니 프로젝트: 할 일 목록 . . . 30
이 장에서 할 일 한눈에 보기 . . . 30
1단계. 새 작업 폴더 만들고 파일 가져오기 (15분) . . . 30
2단계. AGENTS.md 를 이 프로젝트에 맞추기 (10분) . . . 31
3단계. 요구사항과 테스트 기준 대조하기 (20분) . . . 31
4단계. AI 에게 맡기고 계획 검토하기 (25분) . . . 31
5단계. 검증하고 테스트 파일이 바뀌지 않았는지 확인하기 (20분) . . . 32

6단계. 직접 실행해서 확인하기 (15분) . . . 32
7단계. 기록 쓰고 저장하기 (25분) . . . 33
확인 체크리스트 . . . 34
막혔을 때 . . . 34
다음 단계 . . . 34
부록. 템플릿과 예제 . . . 35
A-1. 요구사항 빈칸 틀 (2장) . . . 35
A-2. 작업 지시서 빈칸 틀 (3장) . . . 36
A-3. 규칙 문서 빈칸 틀 (4장) . . . 37
A-4. 권한 설정 완성본 (4장) . . . 38
A-5. 검증 스크립트 (5장) . . . 39
A-6. 커밋 전 검증 훅 (5장) . . . 39
A-7. 할 일 목록 요구사항 (6장) . . . 40
A-8. 할 일 목록 작업 지시서 (6장) . . . 41
A-9. 할 일 목록 테스트 (6장) . . . 42
A-10. 계산기 예제 (4·5장) . . . 43
A-11. 계산기 테스트 (5장) . . . 44

과정 안내

AI 에이전트에게 개발을 맡기되, 규칙·권한·검증으로 통제하고, 통과한 것만 저장하는 법을 단계별로 배웁니다.

코드를 먼저 배운 뒤에 AI 를 쓰는 과정이 아닙니다. AI 에게 맡기면서 코드와 개발 원리를 함께 이해하는 과정입니다. 그래서 각 단계는 "하네스의 한 요소"를 하나씩 익히도록 설계했습니다.

대상	프로그래밍을 처음 배우는 사람 (코딩 경험이 없어도 됩니다)
준비물	Windows PC, Git, OpenCode, OpenRouter 계정 (1단계부터 Python 추가)
필요 없는 것	Docker, 데이터베이스, 웹 서버 (0·1단계 기준)
에이전트	OpenCode (OpenRouter 모델 사용)

단계 안내

단계	대상	내용	상태
0단계	코드를 전혀 모르는 사람	프로그래밍 언어 없이, HTML·CSS 명함 페이지 한 장을 AI 에게 맡기고 하네스의 원리를 익힌다	**이 저장소에 있음** (`docs/stage0/`)
1단계	생초보	Python 계산기와 할 일 목록 프로그램으로, 테스트와 저장 장치까지 익힌다	**이 저장소에 있음** (`docs/stage1/`)
2단계	초급	도서 대출 목록 프로그램으로, 저장, 상태 규칙, 요구사항 변경, 되돌리기를 익힌다	**이 저장소에 있음** (`docs/stage2/`)
3단계	실무형	3일 과정. FastAPI, PostgreSQL, Docker 까지 연결하는 전체 실습	별도 저장소 harness-crud-lab

2단계 교시표

장	주제	하네스 개념	결과물
준비하기	1단계 도구 확인, 작업 폴더 만들기	도구와 실행 환경	설치 확인
1장	도서관 요구사항 쓰기	생활 규칙을 상태로 바꾸기	도서관 요구사항
2장	파일에 저장하기	상태를 파일로 남기기	책 등록, JSON 저장
3장	대출과 반납	작은 지시로 나눠 맡기기, 상태 규칙	대출과 반납 기능
4장	AI 가 만든 실수 찾기	테스트가 잡는 실수와 못 잡는 실수	빠진 약속을 추가한 테스트
5장	요구사항이 바뀌었을 때	기준은 사람이, 구현은 AI 가	변경 요청서, 대출 기간 기록
6장	잘못된 변경을 되돌리기	`restore` 와 `revert`	되돌린 기록
7장	미니 프로젝트: 도서 대출 목록	1~6장 종합	완성된 프로그램, 과정 기록

도커로 실행하기 (선택)

PC 에 Python 과 OpenCode 를 직접 설치하지 않고, 컨테이너 안에서 0·1·2단계를 할 수 있습니다. 방법은 `docs/docker/00_도커로_실행하기.md` 를 봅니다. 편집은 Windows 의 VS Code 에서, 명령은 컨테이너 안에서 합니다.

0단계 교시표

장	주제	결과물
0장	준비하기 (Git, OpenCode, OpenRouter 연결)	설치 확인
1장	AI 와 하네스 체험	조건 없이 시킨 것과 조건을 붙인 것의 비교
2장	말로 된 요구사항 쓰기	명함 요구사항
3장	작업 지시서 쓰기	여섯 칸 지시서
4장	규칙과 권한 세우기	`AGENTS.md`, 권한 설정 확인
5장	체크리스트로 확인하고 저장하기	체크리스트 확인 후 커밋
6장	미니 프로젝트: 자기소개 명함 페이지	HTML·CSS 명함 페이지, 과정 기록

1단계 교시표

교시	주제	배우는 것	결과물
0장	준비하기	Python, Git, OpenCode, OpenRouter 연결	설치 확인 체크리스트
1장	하네스란 무엇인가	같은 부탁을 조건 없이 한 것과 조건을 붙인 것의 차이	비교 기록
2장	요구사항 쓰기	"이렇게 하면 → 이런 결과"로 흐리지 않게 쓰기	계산기 요구사항
3장	작업 지시서 쓰기	목표·범위·하지 말 것·완료 조건 여섯 칸	지시서와 계획 검토
4장	규칙과 권한 세우기	`AGENTS.md`(부탁)와 `opencode.jsonc`(잠금장치)	규칙 문서, 권한 확인
5장	검증하고 저장하기	빨강→초록, 검증을 통과해야 커밋되는 훅	통과한 커밋
6장	미니 프로젝트	할 일 목록을 AI 에게 맡기고 통제하기	할 일 목록 프로그램, 과정 기록

각 장은 **학습 목표 → 개념 → 따라 하기 → 확인하기 → 자주 만나는 오류 → 확인 문제** 순서입니다.

저장소 구조

```
harness-beginner-tutorial/
├ README.md                 이 문서
├ docs/
|   ├ stage0/               0단계 교안 (0장~6장)
|   ├ stage1/               1단계 교안 (0장~6장)
|   ├ docker/               도커로 실행하기 (0·1·2단계 공통)
|   ├ stage2/               2단계 교안 (준비하기, 1장~7장)
|   ├ 하네스_입문_0단계_교안.pdf
|   ├ 하네스_입문_1단계_교안.pdf
|   └ 하네스_입문_2단계_교안.pdf
├ templates/                수업에서 복사해서 쓰는 틀과 파일
|   ├ stage0/               0단계 틀 (명함 요구사항·지시서·규칙·권한·체크리스트)
|   ├ stage2/               2단계 틀 (도서관 요구사항·지시서·변경 요청서·테스트 기준)
|   ├ REQ_빈칸.md            2장: 요구사항 빈칸 틀
|   ├ TASK_빈칸.md           3장: 작업 지시서 빈칸 틀
|   ├ AGENTS_빈칸.md         4장: 규칙 문서 빈칸 틀
|   ├ opencode.jsonc        4장: 권한 설정 (완성본)
|   ├ REQUIREMENTS.md       6장: 할 일 목록 요구사항
|   ├ TASK_todo.md          6장: 할 일 목록 작업 지시서
|   ├ tests/test_todo.py    6장: 할 일 목록 테스트 (AI 가 고칠 수 없는 기준)
|   ├ verify.py             5장: 검증 스크립트
|   ├ pre-commit            5장: 커밋 전 검증 훅
|   └ .gitattributes        줄바꿈 설정
├ docker/                   도커 실행 환경 (0·1·2단계 공용)
|   ├ Dockerfile            Python, pytest, Git, OpenCode 가 든 이미지 설계도
|   ├ docker-compose.yml    작업 폴더와 템플릿 폴더를 연결하는 설정
|   └ .env.example          경로와 Git 이름 설정 견본
├ examples/
|   ├ calc/                 1단계 4·5장에서 쓰는 계산기 예제 (multiply 가 빠진 상태)
|   └ stage2/               2단계 4장에서 쓰는 연습용 초안 (검증 전, 실수 포함)
└ tools/
    └ build_guide.py        교안 PDF 를 만드는 스크립트 (인자: 0 또는 1)
```

시작하기

0단계(`docs/stage0/`), 1단계(`docs/stage1/`), 2단계(`docs/stage2/`) 순서로 따라 합니다.

■ **[PS] 윈도우 PowerShell** 저장소를 내려받습니다.

```
git clone https://github.com/sinaboro/harness-beginner-tutorial C:\harness-beginner-tutorial
```

교안은 PDF 로도 볼 수 있습니다. `docs/하네스_입문_0단계_교안.pdf`, `docs/하네스_입문_1단계_교안.pdf`, `docs/하네스_입문_2단계_교안.pdf` 를 여세요.

0단계부터 순서대로 합니다. 0단계는 Python 이 필요 없고, 1단계부터 Python 을 씁니다.

원칙 세 가지

1. **AI 가 쓰고, 사람이 판단합니다.** 코드는 AI 가 쓰더라도, 계획을 승인하고 결과를 읽고 저장하는 판단은 사람이 합니다.
2. **기준은 AI 가 고치지 않습니다.** 테스트와 검증 스크립트, 규칙 문서는 권한 설정으로 AI 가 고치지 못하게 막습니다.
3. **통과한 것만 저장합니다.** 검증이 실패하면 커밋 자체가 취소됩니다.

강사 안내

- 교안의 실행 결과는 0장~6장의 명령을 실제 Windows PowerShell 에서 확인한 것을 기준으로 합니다.
  OpenCode 의 AI 답변은 실행할 때마다 다르므로, 교안에는 답변 내용 대신 **확인 방법**을 적었습니다.
- 6장에는 정답 코드를 싣지 않았습니다. 학생이 테스트와 지시서를 읽고 AI 와 대화하며 맞춰 가도록 한 것입니다.
- 3단계(3일 과정)는 별도 저장소 harness-crud-lab 에 있습니다. 2단계 7장을 마치면 그 저장소로 이어 갑니다.

1단계 준비하기 (도커)

시간	1시간 30분 (처음 한 번)
준비물	Windows 10·11, Docker Desktop, Git for Windows, VS Code, 인터넷, OpenRouter 계정
결과	도커 컨테이너 안에 Python, pytest, Git, OpenCode 가 준비되고, OpenRouter 에 연결된다

1단계부터 Python 을 씁니다. 그래도 **PC 에 Python 을 설치하지 않습니다.** Python 은 컨테이너 안에 들어 있습니다.

> **역할 나누기**
> - **Windows 쪽(PowerShell)**: 설치하고, 저장소를 받고, 컨테이너를 켠다. `[Windows]` 표시.
> - **컨테이너 쪽(bash)**: 교안의 실습 명령을 친다. `[컨테이너]` 표시.
> - **편집과 확인**: Windows 의 VS Code 에서 파일을 고친다.

1. Docker Desktop 설치 (Windows)

1. https://www.docker.com/products/docker-desktop/ 에서 받아 설치합니다.
2. 설치 중 **Use WSL 2 instead of Hyper-V** 가 보이면 체크된 채로 둡니다.
3. PC 를 재시작하고 Docker Desktop 을 실행합니다. **Engine running** 이 될 때까지 기다립니다.

[Windows] (PowerShell)

```
docker version
```

Client 와 Server 두 줄이 모두 나와야 합니다.

2. 저장소 받기와 설정 파일 (Windows)

[Windows] (PowerShell)

```
git clone https://github.com/sinaboro/harness-beginner-tutorial C:\harness-beginner-tutorial
New-Item -ItemType Directory C:\harness-lab
cd C:\harness-beginner-tutorial\docker
Copy-Item .env.example .env
```

VS Code 로 `C:\harness-beginner-tutorial\docker\.env` 를 열어 `GIT_NAME` 과 `GIT_EMAIL` 을 본인 것으로 바꾸고 저장합니다.

3. 작업 환경 만들기 (Windows, 처음 한 번)

[Windows] (PowerShell)

```
cd C:\harness-beginner-tutorial\docker
docker compose build
```

처음에는 몇 분 걸립니다. 마지막에 `Built` 또는 `Successfully` 가 보이면 됩니다.

4. 컨테이너 들어가기 (Windows)

[Windows] (PowerShell)

```
docker compose run --rm workspace
```

프롬프트가 `root@…:/workspace#` 로 바뀌면 컨테이너 안입니다. 확인합니다.

[컨테이너]

```
pwd
python --version
python -m pytest --version
git --version
opencode --version
```

`/workspace` 는 Windows 의 `C:\harness-lab` 과 같은 폴더이고, `/tutorial` 에는 교안의 템플릿과 예제가 있습니다(읽기 전용).

5. OpenRouter 연결 (컨테이너 안)

[컨테이너]

```
opencode
```

입력 칸에 `/connect` 를 치고 **OpenRouter** 를 고른 뒤 키를 붙여 넣습니다. `/models` 로 목록이 나오면 `/exit` 로 나옵니다.

> 키는 도커 볼륨에 남습니다. 다음에 다시 들어가도 연결은 그대로입니다.

6. 작업 폴더 확인

[컨테이너]

```
mkdir -p /workspace/calc
ls -la /workspace
```

Windows 탐색기에서 `C:\harness-lab\calc` 가 보이면 연결이 된 것입니다.

확인 체크리스트

- ■ `docker version` 에 Server 가 나온다 (Windows)
- ■ 컨테이너 안에서 `python --version` 과 `python -m pytest --version` 이 나온다
- ■ 컨테이너 안에서 `opencode` → `/models` 목록이 나온다
- ■ Windows 의 `C:\harness-lab\calc` 가 보인다

자주 만나는 오류

증상	해결
`error during connect`	Docker Desktop 이 Engine running 이 아니다. 기다리거나 재시작한다.
`/workspace` 가 비어 있다	`.env` 의 `HARNESS_LAB_DIR` 이 틀렸다. 슬래시로 쓰고 폴더가 있는지 본다.
`pytest` 가 파일을 못 찾는다	명령을 친 폴더가 맞는지 `pwd` 로 확인한다. 과제 폴더(`/workspace/calc` 등)에서 실행한다.
`git commit` 이 `Please tell me who you are`	`.env` 의 `GIT_NAME`, `GIT_EMAIL` 을 채우고 컨테이너를 다시 들어간다.

자세한 설명은 도커로 실행하기를 봅니다.

1단계 1장. 하네스란 무엇인가: 같은 부탁, 다른 결과

실행 환경: 도커. 명령 블록은 컨테이너 안(bash)에서 칩니다. `[Windows]` 로 표시된 블록만 Windows PowerShell 에서 칩니다. 파일 편집은 Windows 의 VS Code 에서 합니다. 자세한 설명은 도커로 실행하기를 봅니다.

시간	2시간
준비물	1단계 00 준비하기를 마친 상태 (`opencode` 가 `/models` 목록을 보여 준다)
결과	조건 없이 시킨 결과와 조건을 붙여 시킨 결과를 표로 비교하고, 차이를 한 문장으로 말한다

이 장에서 할 일 한눈에 보기

1. 실험 A 폴더를 만들고, AI 에게 한 줄로 시킨다. (15분)
2. 실험 A 의 결과를 확인하고 표 A 를 채운다. (25분)
3. 실험 B 폴더를 만들고, 조건 세 개를 붙여 시킨다. (25분)
4. 표 B 를 채우고 표 A 와 비교한다. (20분)
5. 조건 세 개가 막으려는 것을 적는다. (15분)

1단계. 실험 A: 조건 없이 한 줄로 시키기 (15분)

[컨테이너]

```
mkdir -p /workspace/lesson01
cd /workspace/lesson01
opencode
```

OpenCode 입력 칸에 아래 문장을 **그대로** 붙여 넣고 Enter 를 누릅니다. 다른 말은 덧붙이지 않습니다.

■ [AI] OpenCode 입력 칸

```
두 수를 더하고 빼는 계산기 파일 calc.py 를 만들어 줘.
```

AI 가 끝낼 때까지 기다립니다. 끝나면 입력 칸에 `/exit` 를 치고 Enter 를 누릅니다.

2단계. 실험 A 결과 확인하고 표 A 채우기 (25분)

[컨테이너]

```
cd /workspace/lesson01
ls -la
python calc.py
```

무엇을: 아래 표의 칸마다 확인 방법대로 보고 적습니다.

항목	확인 방법	내가 적은 답
1. 만들어진 파일 이름은?	`ls -la` 결과의 파일 이름을 모두 적는다	
2. 파일이 몇 개인가?	`ls -la` 결과에서 `.` 과 `..` 을 빼고 센다	
3. `calc.py` 에 나눗셈(`/`)이 있는가?	VS Code 에서 `calc.py` 를 열고 Ctrl+F 로 `/` 를 찾는다. 있으면 "있음"	
4. 0으로 나눌 때를 막는 검사(`== 0`)가 있는가?	VS Code 에서 Ctrl+F 로 `== 0` 을 찾는다. 있으면 "있음"	

항목	확인 방법	내가 적은 답
5. 테스트 파일이 있는가?	`ls -la` 결과에 `tests` 또는 `test_` 이름이 있는지 본다	
6. 시키지 않은 것(나눗셈, 곱셈 등)이 있는가?	3번 결과를 보고 "있음/없음"	
7. AI 가 작업 전에 계획을 말했는가?	대화 화면을 위로 올려 "계획" 단어가 있는지 본다	

확인 기준: `python calc.py` 를 실행하면 아무것도 출력되지 않거나 그냥 끝나야 정상입니다. 오류가 나도 그 오류 문장을 표 밑에 그대로 옮겨 적습니다.

3단계. 실험 B: 조건 세 개를 붙여 시키기 (25분)

[컨테이너]

```
mkdir -p /workspace/lesson01b
cd /workspace/lesson01b
opencode
```

■ [AI] OpenCode 입력 칸

```
두 수를 더하고 빼는 계산기 파일 calc.py 를 만들어 줘.
조건 1. 파일은 calc.py 하나만 만든다. 다른 파일은 만들지 않는다.
조건 2. 더하기(add)와 빼기(subtract) 함수만 만든다. 다른 기능은 추가하지 않는다.
조건 3. 파일을 고치기 전에 계획을 먼저 말하고, 내가 "진행"이라고 할 때까지 기다린다.
```

AI 가 계획을 말하면, 계획에 "나눗셈" 또는 "곱셈"이 있는지 봅니다.

- 있다: 아래 문장을 그대로 입력합니다. `[번호]` 는 계획의 해당 줄 번호로 바꿉니다.
- 없다: 아래 문장만 입력합니다.

■ [AI] OpenCode 입력 칸

```
계획 [번호]번은 조건 2에 어긋나. 그 부분을 빼고 계획을 다시 말해 줘.
```

■ [AI] OpenCode 입력 칸

```
진행
```

끝나면 `/exit` 를 칩니다.

4단계. 표 B 채우고 표 A 와 비교하기 (20분)

[컨테이너]

```
cd /workspace/lesson01b
ls -la
python calc.py
```

표 B 는 표 A 와 **같은 번호, 같은 확인 방법**으로 채웁니다.

항목	표 A (1단계에서 적은 답)	표 B (이번 답)	같은가, 다른가
1. 파일 이름			
2. 파일 개수			
3. 나눗셈(/)이 있는가?			
4. 0 나누기 검사(`== 0`)가 있는가?			
5. 테스트 파일이 있는가?			

항목	표 A (1단계에서 적은 답)	표 B (이번 답)	같은가, 다른가
6. 시키지 않은 것이 있는가?			
7. 계획을 먼저 말했는가?			

비교 기준:

- 같은 번호의 두 칸을 읽고, 같으면 "같음", 다르면 "다름"을 네 번째 칸에 적습니다.
- "다름"이 가장 많은 항목 한 개에 동그라미를 칩니다.

5단계. 조건 세 개가 막으려는 것 적기 (15분)

실험 B 의 조건 세 개가 **각각 무엇을 막으려고 한 것인지** 한 문장으로 적습니다.

조건	막으려는 것 (한 문장)
조건 1 (파일 하나만)	
조건 2 (더하기와 빼기만)	
조건 3 (계획 먼저)	

확인 기준: 각 문장에 "~하지 못하게" 또는 "~이 생기지 않게"의 뜻이 있어야 합니다. 없으면 다시 씁니다.

확인 체크리스트

- ■ 표 A 의 일곱 항목을 모두 채웠다.
- ■ 표 B 의 일곱 항목을 모두 채우고, "같음/다름"을 적었다.
- ■ 다름이 가장 많은 항목에 동그라미를 쳤다.
- ■ 조건 세 개의 막으려는 것을 한 문장씩 적었다.

자주 만나는 오류

증상	해결
`opencode` 가 "모델을 선택하세요"라고 한다	00 준비하기 5번의 `/connect` 를 다시 한다.
실험 B 에서 AI 가 계획 없이 바로 만든다	조건 3 을 바꾸지 않고 다시 입력한다. 이미 만들었다면 `/undo` 를 친다.
`python calc.py` 에서 오류가 난다	오류 문장을 표 밑에 그대로 옮긴다. 2장과 5장에서 이런 오류를 다룬다.

1단계 2장. 요구사항 쓰기: 계산기 요구사항 한 장

실행 환경: 도커. 명령 블록은 컨테이너 안(bash)에서 칩니다. `[Windows]` 로 표시된 블록만 Windows PowerShell 에서 칩니다. 파일 편집은 Windows 의 VS Code 에서 합니다. 자세한 설명은 도커로 실행하기를 봅니다.

시간	2시간
준비물	1장 완료
결과	계산기 요구사항 파일 `요구사항.md` 를 칸 채우기 방식으로 완성하고, 점검 항목을 모두 통과한다

이 장에서 할 일 한눈에 보기

1. 흐린 말을 확인할 수 있는 말로 바꾼다. (15분)
2. 요구사항 파일을 만든다. (5분)
3. 칸을 워크시트대로 채운다. (40분)
4. 점검 항목을 확인하고 고친다. (25분)
5. 옆 사람과 바꿔 읽고 질문을 적는다. (15분)

1단계. 흐린 말 고치기 (15분)

아래 문장에서 흐린 낱말을 찾아 표에 옮기고, 각각을 **눈이나 숫자로 확인할 수 있는 말**로 바꿉니다.

> 계산기는 숫자를 잘 계산하고, 빠르고 정확하며, 사용하기 편해야 한다.

흐린 말	확인할 수 있는 말로 바꾼 것 (예: "2와 3을 더하면 5가 나온다")
잘 계산한다	
빠르고	
정확하며	
사용하기 편해야 한다	

확인 기준: 바꾼 말에 **숫자, 글자, 또는 "나온다/나오지 않는다"** 가 들어 있어야 합니다. 없으면 다시 씁니다.

2단계. 요구사항 파일 만들기 (5분)

[컨테이너]

```
mkdir -p /workspace/req1
cd /workspace/req1
touch 요구사항.md
ls -la
```

[Windows] VS Code 에서 `C:\harness-lab\req1\요구사항.md` 를 엽니다.

3단계. 워크시트대로 칸 채우기 (40분)

무엇을: 요구사항.md 에 아래 틀을 **그대로** 붙여 넣고, [ ] 칸만 채웁니다.

```
# 요구사항: 계산기

## 기능 요구사항

### R1. 더하기
- 이렇게 하면: add(a, b) 를 부른다. a 와 b 는 숫자다.
- 이런 결과가 나온다: [결과 규칙을 한 문장으로. 예: 두 수의 합을 돌려준다. add(2, 3) 은 5 이다.]
- 잘못 쓰면: [잘못된 입력 한 가지와 나오는 결과. 예: 숫자가 아닌 글자를 넣으면 오류가 난다.]

### R2. 빼기
- 이렇게 하면: subtract(a, b) 를 부른다. a 와 b 는 숫자다.
- 이런 결과가 나온다: [결과 규칙. 예: a 에서 b 를 뺀 값을 돌려준다. subtract(10, 4) 는 6 이다.]
- 잘못 쓰면: [잘못된 입력 한 가지와 나오는 결과]

### R3. 나누기
- 이렇게 하면: divide(a, b) 를 부른다. a 와 b 는 숫자다.
- 이런 결과가 나온다: [결과 규칙. 예: a 를 b 로 나눈 값을 돌려준다. divide(9, 3) 은 3 이다.]
- 잘못 쓰면: [b 가 0 일 때 나오는 결과. 예: ValueError 오류를 내고 "0으로 나눌 수 없습니다."라는 글을 보여 준다.]

## 만들지 않는 것
- [이번에 만들지 않을 기능 한 가지. 예: 곱하기]
- [만들지 않을 기능 한 가지. 예: 화면 또는 파일 저장]
```

칸 채우기 규칙 (대괄호 [ ] 칸마다):

칸	무엇을 쓰나	바꾸는 방법
R1 "이런 결과가 나온다"	결과 규칙 한 문장 + 예 한 개	예시 문장을 참고해 내 말로 바꾼다
R1 "잘못 쓰면"	잘못된 입력 한 가지와 결과	예시를 참고해 쓴다
R2 "이런 결과가 나온다"	빼기 규칙 + 예 한 개	예: `subtract(10, 4)` 는 6
R2 "잘못 쓰면"	잘못된 입력 한 가지와 결과	숫자가 아닌 글자를 넣는 경우를 쓴다
R3 "이런 결과가 나온다"	나눗셈 규칙 + 예 한 개	예: `divide(9, 3)` 은 3
R3 "잘못 쓰면"	0으로 나누는 경우의 결과	예시 문장을 그대로 써도 된다
"만들지 않는 것" 두 줄	이번에 만들지 않을 것 두 개	예: 곱하기, 화면

하는 법: VS Code 에서 [를 Ctrl+F 로 찾아, 칸의 [부터]까지를 내 문장으로 바꿉니다. 대괄호도 함께 지웁니다.

4단계. 점검 항목 확인하고 고치기 (25분)

[컨테이너]

```
cd /workspace/req1
grep -c "\[" 요구사항.md
grep -c "잘못 쓰면" 요구사항.md
grep -c "이런 결과가 나온다" 요구사항.md
```

아래 표의 항목을 한 줄씩 확인합니다.

번호	확인할 것	확인 방법	예 / 아니오
1	[가 한 개도 없다	첫 명령 결과가 `0`	
2	"잘못 쓰면"이 세 번 나온다 (R1~R3)	둘째 명령 결과가 `3`	

번호	확인할 것	확인 방법	예 / 아니오
3	"이런 결과가 나온다"가 세 번 나온다	셋째 명령 결과가 `3`	
4	"잘 된다", "빠르다", "정확하다", "편하다"가 없다	`grep -c "잘 |빠르|정확|편하" 요구사항.md` 결과가 `0`	
5	"만들지 않는 것" 아래에 두 줄이 있다	VS Code 에서 그 아래 줄을 세어 본다	

아니오가 있으면, 그 줄을 VS Code 에서 고치고 같은 명령을 다시 칩니다. 고친 번호는 옆에 적습니다.

5단계. 옆 사람과 바꿔 읽고 질문 적기 (15분)

옆 사람의 `요구사항.md` 를 읽고 질문을 두 개 이상 적습니다. 형식은 아래와 같습니다.

```
R[번호] 에서 [무엇]이 궁금합니다.
```

질문에 답이 적혀 있지 않으면 그 줄이 빈틈입니다. 빈틈이 있는 줄을 한 문장 더 구체적으로 고칩니다.

확인 체크리스트

- ■ 흐린 말 네 개를 확인할 수 있는 말로 바꿨다.
- ■ `요구사항.md` 에 `[` 가 0건이다.
- ■ 4단계의 다섯 항목을 모두 확인하고, 아니오는 고쳤다.
- ■ 옆 사람 파일에서 질문을 두 개 이상 적었다.

자주 만나는 오류

증상	해결
`grep -c "\["` 결과가 0이 아니다	`grep -n "\[" 요구사항.md` 로 줄 번호를 본다. 그 줄의 대괄호를 지운다.
"잘못 쓰면" 개수가 3이 아니다	빠진 기능의 "잘못 쓰면" 줄을 추가하거나, 중복된 줄을 지운다.
예시 문장을 그대로 복사했다	예시는 참고용이다. 내 말로 한 번 바꾼다. 그래도 같으면 괜찮다.

1단계 3장. 작업 지시서 쓰기

실행 환경: 도커. 명령 블록은 컨테이너 안(bash)에서 칩니다. `[Windows]` 로 표시된 블록만 Windows PowerShell 에서 칩니다. 파일 편집은 Windows 의 VS Code 에서 합니다. 자세한 설명은 도커로 실행하기를 봅니다.

시간	2시간
준비물	2장의 요구사항.md (/workspace/req1)
결과	지시서 지시서.md 를 여섯 칸 워크시트로 완성하고, AI 의 계획을 검토해 승인한다. 결과를 지시서와 대조한다

이 장에서 할 일 한눈에 보기

1. 지시서 파일을 만든다. (5분)
2. 여섯 칸을 워크시트대로 채운다. (35분)
3. AI 에게 지시서를 건네고 계획을 받는다. (15분)
4. 계획을 네 질문으로 검토하고 승인한다. (20분)
5. 결과를 지시서와 대조한다. (20분)
6. 1장 실험 A 와 비교표를 채운다. (15분)

1단계. 지시서 파일 만들기 (5분)

[컨테이너]

```
mkdir -p /workspace/task1
cd /workspace/task1
touch 지시서.md
ls -la
```

[Windows] VS Code 에서 `C:\harness-lab\task1\지시서.md` 를 엽니다.

2단계. 여섯 칸 워크시트 채우기 (35분)

무엇을: 지시서.md 에 아래 틀을 **그대로** 붙여 넣고, `[ ]` 칸을 채웁니다.

```
# 작업 지시서: 계산기 만들기

## 1. 목표
[끝나면 무엇이 되어 있어야 하는지 한 문장]

## 2. 요구사항
[2장 요구사항.md 에서 만들 번호. 예: R1, R2]

## 3. 범위
- 고쳐도 되는 파일: [파일 이름 한 개]
- 고치면 안 되는 파일: [파일 이름 두 개. 예: 요구사항.md, 지시서.md]

## 4. 하지 말 것
- [하지 말아야 할 것 한 가지]
- [하지 말아야 할 것 한 가지]
- [하지 말아야 할 것 한 가지]

## 5. 완료 조건
- [ ] [확인할 수 있는 조건 한 가지]
```

```
- [ ] [확인할 수 있는 조건 한 가지]

## 6. 진행 방법
먼저 계획만 말한다. 내가 "진행"이라고 하기 전에는 파일을 만들거나 고치지 않는다.
끝나면 바꾼 파일 이름과 완료 조건을 확인한 결과를 보고한다.
```

칸별 워크시트 (아래 예를 참고해 **내 말로** 바꿉니다):

칸	무엇을 쓰나	예
1. 목표	파일 한 개와 함수 목록	calc.py 에 더하기(add)와 빼기(subtract) 함수를 만든다
2. 요구사항	2장의 번호	R1, R2
3. 고쳐도 되는 파일	파일 이름 한 개	calc.py
3. 고치면 안 되는 파일	파일 이름 두 개	요구사항.md, 지시서.md
4. 하지 말 것	2장 "만들지 않는 것"에서 세 개 골라 쓴다	곱하기 함수를 만들지 않는다 / 화면을 만들지 않는다 / 파일을 저장하지 않는다
5. 완료 조건 (첫 줄)	눈으로 볼 수 있는 조건	`python calc.py` 를 실행하면 오류 없이 끝난다
5. 완료 조건 (둘째 줄)	요구사항의 결과 한 개	add(2, 3) 을 부르면 5가 나온다

확인 기준: 다음 세 가지를 보고, 아니오가 있으면 고칩니다.

- 6칸 중 `[` 가 남은 칸이 없다. (`grep -c "\[" 지시서.md` 결과가 `0`)
- 5번 완료 조건이 "잘 된다"처럼 흐리지 않다. 결과를 보고 맞다/아니다를 말할 수 있다.
- 4번 하지 말 것에 적은 내용이 2장 "만들지 않는 것"에 있다.

3단계. AI 에게 지시서 건네기 (15분)

[컨테이너]

```
cd /workspace/task1
opencode
```

OpenCode 입력 칸에 아래 문장을 **그대로** 붙여 넣습니다.

[AI] OpenCode 입력 칸

```
지시서.md 를 읽고 그 지시대로 작업해 줘. 먼저 계획만 말하고, 내가 "진행"이라고 할 때까지 파일을 만들거나 고치지 마.
```

AI 가 계획을 말하면 4단계로 갑니다. 이 시점에는 "진행"을 입력하지 않습니다.

4단계. 계획 검토하고 승인하기 (20분)

무엇을: AI 의 계획을 아래 네 질문으로 검토합니다.

질문	답 (예/아니오)	아니오면 할 일
1. 고치는 파일이 calc.py 한 개뿐인가?		아래 문장 1번
2. 계획에 곱하기, 나누기, 파일 저장이 없는가?		아래 문장 2번
3. 계획이 요구사항.md 나 지시서.md 를 고친다고 하지 않는가?		아래 문장 3번
4. 계획에 더하기와 빼기가 모두 있는가?		아래 문장 4번

아니오가 있으면 아래 문장을 입력하고, 번호만 바꿉니다.

■ [AI] OpenCode 입력 칸

> 계획 중 [번호]번을 빼고, 다시 계획을 말해 줘. 아직 파일은 만들지 마.

모두 예이면 입력합니다.

■ [AI] OpenCode 입력 칸

> 진행

끝나면 `/exit` 를 칩니다.

5단계. 결과를 지시서와 대조하기 (20분)

[컨테이너]

```
cd /workspace/task1
ls -la
python calc.py
grep -c "def add\|def subtract" calc.py
grep -c "def multiply\|def divide" calc.py
```

무엇을: 아래 표를 채웁니다.

지시서의 항목	실제 결과 (위 명령의 출력)	같은가?
3번 고쳐도 되는 파일: calc.py 한 개	`ls -la` 에 파일이 calc.py 만 새로 생겼는가?	
4번 하지 말 것: 곱하기, 나누기 없음	둘째 `grep -c` 결과가 `0` 인가?	
5번 완료 조건: 첫 줄	`python calc.py` 가 오류 없이 끝났는가?	
5번 완료 조건: 둘째 줄	VS Code 에서 add 함수 설명대로 계산이 되는지 본다	

확인 기준: 표의 "같은가?" 칸이 모두 "예"여야 합니다. "아니오"가 있으면 AI 에게 그 항목 번호를 알리고 고치게 합니다.

■ [AI] OpenCode 입력 칸

> 지시서 [번호]번과 실제 결과가 달라. [실제로 본 것 한 문장]이야. 원인을 먼저 설명하고, 내가 "진행"이라고 하면 calc.py 만 고쳐 줘.

6단계. 1장 실험 A 와 비교하기 (15분)

항목	1장 실험 A (한 줄 지시)	3장 (지시서)
계획을 먼저 말했는가?		
시키지 않은 기능이 있었는가?		
완료를 확인할 방법이 있었는가?		
결과를 지시서와 대조할 수 있었는가?		

비교 기준: 같은 항목의 두 칸에 "예/아니오"로 적습니다. 설명은 한 단어로 줄입니다.

확인 체크리스트

- ■ `지시서.md` 의 여섯 칸을 모두 채웠다. `[` 가 0건이다.
- ■ 계획 검토 네 질문에 모두 답했다.
- ■ 5단계 표의 네 항목을 대조했다.
- ■ 6단계 비교표를 채웠다.

자주 만나는 오류

증상	해결
AI 가 `지시서.md` 를 찾지 못한다	`cd /workspace/task1` 을 하고 `opencode` 를 쳤는지 확인한다.
AI 가 "진행"을 기다리지 않고 만든다	처음 입력한 문장에 "먼저 계획만 말하고"가 있었는지 확인한다. 이미 만들었다면 `/undo` 를 친다.
`python calc.py` 에서 오류가 난다	오류 문장을 적고, AI 에게 그 문장을 붙여 원인부터 설명하게 한다.

1단계 4장. 규칙과 권한 세우기

실행 환경: 도커. 명령 블록은 컨테이너 안(bash)에서 칩니다. `[Windows]` 로 표시된 블록만 Windows PowerShell 에서 칩니다. 파일 편집은 Windows 의 VS Code 에서 합니다. 자세한 설명은 도커로 실행하기를 봅니다.

시간	2시간
준비물	3장 완료
결과	규칙 파일 `AGENTS.md` 의 `[작성]` 칸 17개를 정해진 문장으로 채우고, 권한 설정이 막는 일과 묻는 일을 직접 확인한다

이 장에서 할 일 한눈에 보기

1. 작업 폴더를 만들고 계산기 예제와 설정 파일을 가져온다. (10분)
2. `AGENTS.md` 의 `[작성]` 17개를 표의 문장으로 채운다. (35분)
3. 권한 설정 파일의 규칙 세 개를 읽고 답한다. (15분)
4. AI 에게 막혀야 할 일을 시키고 결과를 적는다. (25분)
5. AI 가 규칙을 읽었는지 확인한다. (10분)

1단계. 작업 폴더 만들기 (10분)

[컨테이너]

```
mkdir -p /workspace/calc
cp -r /tutorial/examples/calc/. /workspace/calc/
cd /workspace/calc
cp /tutorial/templates/verify.py .
cp /tutorial/templates/opencode.jsonc .
cp /tutorial/templates/AGENTS_빈칸.md AGENTS.md
git init
ls -la
```

`ls -la` 결과에 아래 여섯 개가 모두 있어야 합니다. 없으면 빠진 이름을 적고 그 `cp` 명령을 다시 칩니다.

파일	있는가? (예/아니오)
calc.py	
tests (폴더)	
pytest.ini	
verify.py	
opencode.jsonc	
AGENTS.md	

2단계. AGENTS.md 의 [작성] 17개 채우기 (35분)

무엇을: Windows 의 VS Code 에서 `C:\harness-lab\calc\AGENTS.md` 를 엽니다.

먼저 한 번만 할 일: `todo.py` 라는 글자를 `calc.py` 로 바꿉니다. Ctrl+H 를 누르고, 찾을 내용 `todo.py`, 바꿀 내용 `calc.py` 를 넣은 뒤 **모두 바꾸기**를 누릅니다.

아래 표는 파일에 나오는 [작성] 순서대로입니다. **각 칸을 찾아 오른쪽 문장으로 바꿉니다.** 문장은 한 글자도 바꾸지 않고 그대로 씁니다. 칸 뒤의 괄호 설명(예: (먼저 무엇을 하는가))은 지웁니다.

번호	파일에서 찾을 칸 (Ctrl+F 로 검색)	그대로 써야 할 문장
①	`- 목표: [작성]`	두 수의 더하기, 빼기, 곱하기, 나누기 함수를 calc.py 에 만든다.
②	`- 사용자: [작성]`	프로그래밍을 처음 배우는 사람
③	`¦ calc.py ¦ [작성] ¦` (처음 `todo.py` 였던 줄)	고쳐도 된다
④	`¦ tests/ ¦ [작성] ¦`	고치면 안 된다
⑤	`¦ AGENTS.md, opencode.jsonc, verify.py ¦ [작성] ¦`	고치면 안 된다
⑥	`1. [작성]`	지시서를 읽고 작업 범위(고칠 파일, 하지 말 것)를 확인한다.
⑦	`2. [작성]`	계획을 먼저 말하고, 사람이 "진행"이라고 할 때까지 파일을 고치지 않는다.
⑧	`3. [작성]`	요구사항에 있는 함수만 calc.py 에 만든다. 함수마다 한국어 설명을 단다.
⑨	`4. [작성]`	python verify.py 로 테스트를 실행해서 결과를 확인한다.
⑩	`5. [작성]`	바꾼 파일 이름과 검증 결과를 보고한다.
⑪	`- [작성]` (커밋과 저장)	저장(git commit)과 올리기(git push)는 하지 않는다. 저장은 사람이 한다.
⑫	`- [작성]` (테스트)	tests/ 폴더의 테스트 기대값을 바꾸지 않는다.
⑬	`- [작성]` (새 패키지, 인터넷)	새 패키지를 설치하거나 인터넷에서 자료를 가져오지 않는다.
⑭	`- [작성]` (주석)	파일의 모든 함수 위에 한국어 설명을 단다.
⑮	`- [작성]` (이름 짓기)	이름은 뜻이 드러나는 영어로 짓고, 한 줄에는 한 가지 일만 쓴다.
⑯	`명령을 실행한다` 앞의 `[작성]`	python verify.py 를 실행한다
⑰	`[작성] (몇 번까지 고쳐 보고, ...)`	검증이 실패하면 원인을 설명하고 세 번까지만 고친다. 그래도 안 되면 멈추고 보고한다.

⑯ **줄 설명:** 원문은 `- 완료라고 말하기 전에 [작성] 명령을 실행한다.` 입니다. `[작성]` 을 `python verify.py` 로 바꾸면 됩니다.

채우는 법:

1. Ctrl+F 로 `[작성]` 을 검색합니다. 처음 나오는 칸이 표의 몇 번인지 찾습니다.
2. 그 칸을 표의 문장으로 바꿉니다.
3. 다음 `[작성]` 을 검색해 같은 방법으로 바꿉니다.

확인 기준 (다섯 가지): 아래를 Ctrl+F 로 확인하고, 모두 맞으면 체크합니다.

- ■ `계획을 먼저 말하고, 사람이 "진행"이라고 할 때까지 파일을 고치지 않는다.` 가 들어 있다.
- ■ `tests/ 폴더의 테스트 기대값을 바꾸지 않는다.` 가 들어 있다.

■ `저장(git commit)과 올리기(git push)는 하지 않는다.` 가 들어 있다.

■ `python verify.py` 가 들어 있다.

■ Ctrl+F 로 `[작성]` 을 검색하면 **0건**이다.

[컨테이너]

```
cd /workspace/calc
grep -c "\[작성\]" AGENTS.md
grep -c "todo" AGENTS.md
```

확인 기준: 첫 명령 결과가 `0`, 둘째 명령 결과가 `0` 이어야 합니다.

3단계. 권한 설정 읽고 답하기 (15분)

무엇을: Windows 의 VS Code 에서 `C:\harness-lab\calc\opencode.jsonc` 를 엽니다. Ctrl+F 로 아래 세 줄을 찾아 읽고, 답을 적습니다.

번호	찾을 줄	이 규칙이 막거나 묻는 것 (한 문장)
1	`"resource": "git commit *", "effect": "deny"`	
2	`"resource": "*tests*", "effect": "deny"` 또는 `tests` 가 들어간 `edit` 규칙	
3	`"resource": "*", "effect": "ask"` (shell 규칙)	

확인 기준: 1번 답에는 "실행할 수 없다", 2번 답에는 "고칠 수 없다", 3번 답에는 "묻는다"가 들어 있어야 합니다.

4단계. 막혀야 할 일을 AI 에게 시키고 결과 적기 (25분)

[컨테이너]

```
cd /workspace/calc
printf 'TEST_SECRET=연습용-값입니다\n' > .env
opencode
```

OpenCode 입력 칸에 아래 세 문장을 **하나씩** 입력합니다. 한 문장을 넣고 결과를 적은 뒤 다음으로 갑니다.

■ **[AI] OpenCode 입력 칸**

```
.env 파일을 읽고 안에 무슨 변수가 있는지 알려 줘.
```

■ **[AI] OpenCode 입력 칸**

```
git commit -m "test" 를 실행해 줘.
```

■ **[AI] OpenCode 입력 칸**

```
tests/test_calc.py 의 첫 줄에 주석 한 줄을 추가해 줘.
```

무엇을: 각 문장에 AI 가 무엇을 했는지 아래 표에 적습니다. 판정 기준은 세 가지입니다. "실행했다", "거절했다", "물어봤다".

시킨 일	AI 가 한 일 (실행했다 / 거절했다 / 물어봤다)	파일이 바뀌었는가? (예/아니오)
.env 읽기		
git commit 실행		
tests/test_calc.py 고치기		

`/exit` 를 친 뒤 확인합니다.

[컨테이너]

```
cd /workspace/calc
git log --oneline
git status
diff tests/test_calc.py /tutorial/examples/calc/tests/test_calc.py && echo "테스트 파일은 원본과 같습니다"
```

확인 기준:

- `git log --oneline` 에 커밋이 없어야 합니다. ("does not have any commits yet" 이 정상)
- `diff` 가 아무것도 출력하지 않고 "테스트 파일은 원본과 같습니다"가 나와야 합니다.
- 둘 중 하나라도 다르면 표의 해당 행을 "예"로 적고, AI 에게 원래대로 되돌리게 합니다. (`cp /tutorial/examples/calc/tests/test_calc.py tests/` 로 되돌린다)

5단계. AI 가 규칙을 읽었는지 확인하기 (10분)

■ **[AI] OpenCode 입력 칸**

```
이 프로젝트에서 작업할 때 지켜야 할 규칙을 파일을 고치지 말고 말로만 알려 줘.
```

AI 의 답에 아래 세 가지가 있는지 확인합니다.

■ "계획을 먼저 말한다" 또는 "진행을 기다린다"

■ "tests 를 고치지 않는다"

■ "저장하지 않는다" 또는 "git commit 을 하지 않는다"

셋 중 하나라도 없으면 `/exit` 후 `opencode` 를 다시 열고 같은 질문을 합니다. 그래도 없으면 2단계의 확인 기준을 다시 봅니다.

확인 체크리스트

■ `AGENTS.md` 에 `[작성]` 이 0건이고, `todo` 가 0건이다.

■ 2단계의 다섯 확인 기준을 모두 체크했다.

■ 3단계 세 답을 적었다.

■ 4단계 표를 채웠고, 테스트 파일이 원본과 같다.

■ 5단계 세 항목을 확인했다.

자주 만나는 오류

증상	해결
`git init` 이 "Reinitialized" 라고 나온다	이미 저장소라는 뜻이다. 무시한다.
AI 가 AGENTS.md 를 언급하지 않는다	`ls -la` 로 AGENTS.md 가 `/workspace/calc` 에 있는지 확인한다.
`git commit` 이 실제로 실행되어 커밋이 생겼다	권한이 막지 못했다는 뜻이다. 표에 "실행했다"라고 적고 강사에게 알린다.
`.env` 가 Git 에 올라갈까 걱정된다	`echo ".env" > .gitignore` 를 한 번 쳐서 막는다. (5장에서 다시 한다)

1단계 5장. 검증하고 저장하기

실행 환경: 도커. 명령 블록은 컨테이너 안(bash)에서 칩니다. `[Windows]` 로 표시된 블록만 Windows PowerShell 에서 칩니다. 파일 편집은 Windows 의 VS Code 에서 합니다. 자세한 설명은 도커로 실행하기를 봅니다.

시간	2시간
준비물	4장의 `/workspace/calc` 폴더 (AGENTS.md, opencode.jsonc, verify.py 있음)
결과	테스트가 빨강에서 초록으로 바뀌는 것을 보고, 검증을 통과해야만 커밋되도록 장치를 켠다

이 장에서 할 일 한눈에 보기

1. 정리하고 빨강을 확인한다. (15분)
2. AI 에게 `multiply` 함수를 추가하게 한다. (30분)
3. 초록을 확인하고 테스트 파일이 바뀌지 않았는지 대조한다. (15분)
4. 커밋 전에 검증하는 장치(훅)를 켠다. (10분)
5. 일부러 틀리게 고쳐 보고, 커밋이 막히는지 본다. (20분)
6. 되돌리고 정식으로 저장한다. (20분)

1단계. 정리하고 빨강 확인하기 (15분)

[컨테이너]

```
cd /workspace/calc
rm -f .env
echo ".env" > .gitignore
cp /tutorial/examples/calc/tests/test_calc.py tests/test_calc.py
git status
python -m pytest
```

확인 기준:

- `git status` 에 `.gitignore` 와 파일들이 보이고, `.env` 는 보이지 않아야 합니다.
- `python -m pytest` 의 출력에 `ERROR` 와 `ImportError` 가 있어야 합니다. 그 줄을 아래에 그대로 적습니다.

```
(여기에 ImportError 줄을 옮겨 적는다)
```

읽는 법:

- `ERROR tests/test_calc.py` → 어느 파일에서 실패했나?
- `cannot import name 'multiply'` → 무엇이 없어서 실패했나?

테스트 파일은 `multiply` 를 쓰는 약속을 가지고 있지만, `calc.py` 에는 아직 `multiply` 가 없습니다. 이것이 **빨강**입니다.

`python -m pytest` 와 `python verify.py` 는 같은 테스트를 실행합니다. `verify.py` 는 마지막에 "검증 통과" 또는 "검증 실패"를 한 줄로 알려 줍니다.

2단계. AI 에게 multiply 추가시키기 (30분)

[컨테이너]

```
cd /workspace/calc
opencode
```

OpenCode 입력 칸에 아래 문장을 **그대로** 입력합니다.

■ [AI] OpenCode 입력 칸

```
calc.py 에 multiply(a, b) 함수를 추가해 줘. 두 수를 곱한 결과를 돌려준다.
규칙:
1. 먼저 계획만 말하고, 내가 "진행"이라고 할 때까지 파일을 고치지 마.
2. tests/ 폴더와 verify.py 는 절대 고치지 마.
3. 끝나면 python verify.py 를 실행해서 결과를 보고해 줘.
```

AI 의 계획을 아래 세 질문으로 검토합니다.

질문	답 (예/아니오)	아니오면 입력할 문장
1. 고치는 파일이 calc.py 한 개뿐인가?		아래 문장 A
2. 계획에 tests/ 나 verify.py 를 고친다는 말이 없는가?		아래 문장 A
3. 계획에 multiply 함수 추가가 들어 있는가?		아래 문장 A

[AI] 문장 A (아니오일 때만, 번호만 바꿔 입력합니다)

```
계획 [번호]번이 규칙과 맞지 않아. 그 부분을 빼고 계획을 다시 말해 줘. 아직 파일은 고치지 마.
```

모두 예이면 입력합니다.

■ [AI] OpenCode 입력 칸

```
진행
```

끝나면 `/exit` 를 칩니다.

3단계. 초록 확인하고 테스트가 바뀌지 않았는지 대조하기 (15분)

[컨테이너]

```
cd /workspace/calc
python -m pytest
python verify.py
diff tests/test_calc.py /tutorial/examples/calc/tests/test_calc.py && echo "테스트 파일은 원본과 같습니다"
grep -n "def multiply" calc.py
```

확인 기준:

확인할 것	기대하는 결과	실제 결과 (예/아니오)
테스트 개수와 결과	`5 passed` 가 나온다	
검증 결과	마지막 줄이 "검증 통과"다	
테스트 파일 대조	"테스트 파일은 원본과 같습니다"가 나온다	
함수 확인	`def multiply` 가 한 줄 나온다	

아니오가 있으면 AI 에게 그 항목을 알리고 고치게 합니다. 테스트 파일이 바뀌었다면, 원본을 다시 복사합니다. `cp /tutorial/examples/calc/tests/test_calc.py tests/`

VS Code 에서 확인: `calc.py` 를 열어 `multiply` 함수 위에 한국어 설명이 있는지 봅니다. 없으면 AI 에게 "multiply 함수 위에 한국어 설명을 달아 줘"라고 말합니다.

4단계. 커밋 전에 검증하는 장치 켜기 (10분)

[컨테이너]

```
cd /workspace/calc
mkdir -p .githooks
cp /tutorial/templates/pre-commit .githooks/pre-commit
cp /tutorial/templates/.gitattributes .
git config core.hooksPath .githooks
git config core.hooksPath
```

확인 기준: 마지막 명령이 `.githooks` 를 출력해야 합니다. 다른 것이 나오면 `git config core.hooksPath .githooks` 를 다시 칩니다.

5단계. 일부러 틀리게 고쳐서 저장이 막히는지 보기 (20분)

[Windows] VS Code 에서 `C:\harness-lab\calc\calc.py` 를 엽니다. Ctrl+H 로 `return a / b` 를 찾아 `return a + b` 로 바꾸고(**한 번만**, 나눗셈 함수 안의 줄), 저장합니다.

[컨테이너]

```
cd /workspace/calc
git add -A
git commit -m "실험: 틀린 나눗셈 저장 시도"
```

확인 기준: 출력에 아래 세 줄이 있어야 합니다.

- `[pre-commit] 저장하기 전에 검증을 실행합니다...`
- `FAILED tests/test_calc.py::test_divide`
- `[pre-commit] 검증 실패: 커밋을 취소했습니다.`

커밋이 생기지 않았는지 확인합니다.

[컨테이너]

```
git log --oneline
```

"does not have any commits yet" 이 나오면 정상입니다(아직 첫 커밋 전).

[Windows] 다시 VS Code 에서 `return a + b` 를 **`return a / b`** 로 되돌리고 저장합니다.

[컨테이너]

```
python verify.py
```

"검증 통과"가 나와야 다음 단계로 갑니다.

6단계. 정식으로 저장하기 (20분)

[컨테이너]

```
cd /workspace/calc
git add -A
git status
git commit -m "5장: multiply 추가, 테스트 통과 (검증 통과 후 저장)"
git log --oneline
```

확인 기준:

- `git status` 에 `.env` 가 없어야 합니다.
- `git log --oneline` 에 커밋이 한 줄 생겨야 합니다.
- 커밋 과정에서 `[pre-commit]` 이 "검증 통과"를 출력해야 합니다.

확인 체크리스트

- ■ 빨강(ImportError)의 줄을 옮겨 적었다.
- ■ `python verify.py` 가 "검증 통과"로 끝났다.
- ■ 테스트 파일이 원본과 같다. (`diff` 가 아무것도 출력하지 않는다)
- ■ 틀린 나눗셈을 커밋하려 했을 때 취소되는 것을 봤다.
- ■ 되돌린 뒤 정식 커밋이 한 줄 생겼다.

자주 만나는 오류

증상	해결
`No module named pytest`	컨테이너를 다시 들어간다. 그래도 안 되면 `docker compose build` 를 Windows 에서 다시 한다.
`[pre-commit]` 이 나오지 않는다	`git config core.hooksPath` 가 `.githooks` 를 출력하는지 본다.
훅이 `python: command not found`	컨테이너 안에서 커밋했는지 확인한다. Windows PowerShell 에서 커밋하면 안 된다.
`LF will be replaced by CRLF` 경고	기능에는 문제가 없다. 4단계에서 `.gitattributes` 를 복사했으므로 사라진다.
`git commit` 이 `Please tell me who you are`	`.env` 의 `GIT_NAME`, `GIT_EMAIL` 을 채우고 컨테이너를 다시 들어간다.

1단계 6장. 미니 프로젝트: 할 일 목록

실행 환경: 도커. 명령 블록은 컨테이너 안(bash)에서 칩니다. `[Windows]` 로 표시된 블록만 Windows PowerShell 에서 칩니다. 파일 편집은 Windows 의 VS Code 에서 합니다. 자세한 설명은 도커로 실행하기를 봅니다.

시간	2시간 30분
준비물	1~5장 완료
결과	할 일 목록 프로그램 `todo.py` 를 AI 에게 맡기고, 테스트와 대조한 뒤 저장한다. 과정 기록을 쓴다

이 장은 **정답 파일을 주지 않습니다.** 요구사항과 테스트를 기준으로 확인합니다.

이 장에서 할 일 한눈에 보기

1. 새 작업 폴더를 만들고 파일을 가져온다. (15분)
2. 규칙 파일의 `calc.py` 를 `todo.py` 로 바꾼다. (10분)
3. 요구사항과 테스트 기준을 대조한다. (20분)
4. AI 에게 맡기고 계획을 검토한다. (25분)
5. 검증하고, 테스트 파일이 바뀌지 않았는지 확인한다. (20분)
6. 직접 실행해서 확인한다. (15분)
7. 기록을 쓰고 저장한다. (25분)

1단계. 새 작업 폴더 만들고 파일 가져오기 (15분)

[컨테이너]

```
mkdir -p /workspace/my-todo/tests /workspace/my-todo/.githooks
cd /workspace/my-todo
cp /tutorial/templates/tests/test_todo.py tests/
cp /tutorial/templates/REQUIREMENTS.md .
cp /tutorial/templates/TASK_todo.md .
cp /tutorial/templates/verify.py .
cp /tutorial/templates/opencode.jsonc .
cp /tutorial/templates/pre-commit .githooks/pre-commit
cp /tutorial/templates/.gitattributes .
cp /tutorial/examples/calc/pytest.ini .
cp /workspace/calc/AGENTS.md .
echo ".env" > .gitignore
git init
git config core.hooksPath .githooks
ls -la
ls tests
```

확인 기준: 아래 표의 파일이 모두 있어야 합니다.

파일	있는가? (예/아니오)
REQUIREMENTS.md	
TASK_todo.md	
AGENTS.md	

파일	있는가? (예/아니오)
opencode.jsonc	
verify.py	
pytest.ini	
tests/test_todo.py	
.githooks/pre-commit	

`git config core.hooksPath` 가 `.githooks` 를 출력해야 합니다.

2단계. AGENTS.md 를 이 프로젝트에 맞추기 (10분)

무엇을: 4장에서 만든 `AGENTS.md` 는 `calc.py` 를 가리킵니다. 이 프로젝트의 파일 이름 `todo.py` 로 바꿉니다.

[Windows] VS Code 에서 `C:\harness-lab\my-todo\AGENTS.md` 를 열고, Ctrl+H 로 `calc.py` 를 `todo.py` 로 **모두 바꾸기** 합니다.

[컨테이너]

```
cd /workspace/my-todo
grep -c "calc" AGENTS.md
grep -c "todo.py" AGENTS.md
```

확인 기준: 첫 명령 결과가 `0`, 둘째 명령 결과가 `1` 이상이어야 합니다.

3단계. 요구사항과 테스트 기준 대조하기 (20분)

무엇을: `REQUIREMENTS.md` 의 함수 이름과 `tests/test_todo.py` 의 함수 이름이 같은지 확인합니다.

[컨테이너]

```
cd /workspace/my-todo
grep -n "^### R\|add_task\|complete_task\|delete_task\|save_tasks\|load_tasks" REQUIREMENTS.md
grep -n "^from todo import" tests/test_todo.py
```

아래 표를 채웁니다.

요구사항	함수 이름 (REQUIREMENTS.md)	테스트가 부르는가? (tests/test_todo.py 에서 찾는다)
R1 할 일 추가	add_task	
R2 완료 표시	complete_task	
R3 삭제	delete_task	
R4 저장과 읽기	save_tasks, load_tasks	

확인 기준: 네 줄 모두 "예"여야 합니다. 아니오가 있으면 그 함수가 테스트에 없다는 뜻입니다. `REQUIREMENTS.md` 를 고치지 말고, 강사에게 알립니다.

4단계. AI 에게 맡기고 계획 검토하기 (25분)

[컨테이너]

```
cd /workspace/my-todo
opencode
```

OpenCode 입력 칸에 아래 문장을 **그대로** 입력합니다.

■ [AI] OpenCode 입력 칸

```
TASK_todo.md 를 읽고 그 지시대로 작업해 줘. REQUIREMENTS.md 와 tests/ 를 기준으로 삼는다. 먼저 계획만 말하고 내 승인을 기다려.
```

계획을 아래 네 질문으로 검토합니다.

질문	답 (예/아니오)	아니오면 입력할 문장
1. 고치는 파일이 todo.py 한 개뿐인가?		아래 문장 A
2. 계획에 tests/, verify.py, AGENTS.md 를 고친다는 말이 없는가?		아래 문장 A
3. 계획에 R1~R4 의 함수가 모두 들어 있는가?		아래 문장 A
4. 계획에 웹 화면, 데이터베이스, 날짜, 검색이 없는가?		아래 문장 A

[AI] 문장 A (아니오일 때, 번호만 바꿔 입력합니다)

```
계획 [번호]번이 TASK_todo.md 의 범위나 "하지 말 것"과 맞지 않아. 그 부분을 고쳐서 계획을 다시 말해 줘. 아직 파일은 만들지 마.
```

모두 예이면 입력합니다.

■ [AI] OpenCode 입력 칸

```
진행
```

끝나면 `/exit` 를 칩니다.

5단계. 검증하고 테스트 파일이 바뀌지 않았는지 확인하기 (20분)

[컨테이너]

```
cd /workspace/my-todo
python verify.py
python -m pytest -q
diff tests/test_todo.py /tutorial/templates/tests/test_todo.py && echo "테스트 파일은 원본과 같습니다"
git status
```

확인 기준:

확인할 것	기대하는 결과	실제 결과 (예/아니오)
검증 결과	마지막 줄이 "검증 통과"다	
테스트 파일	"테스트 파일은 원본과 같습니다"가 나온다	
바뀐 파일	`git status` 의 새 파일이 todo.py 뿐이다 (AGENTS.md 등 다른 파일은 없다)	

아니오가 있으면 AI 에게 그 항목을 알리고 고치게 합니다. 테스트 파일이 바뀌었다면 원본을 다시 복사합니다. `cp /tutorial/templates/tests/test_todo.py tests/`

AI 에게 알리는 문장:

■ [AI] OpenCode 입력 칸

```
python verify.py 결과가 [실패 줄 한 문장]이야. 원인을 먼저 설명하고, 내가 "진행"이라고 하면 todo.py 만 고쳐 줘. tests/ 는 고치지 마.
```

6단계. 직접 실행해서 확인하기 (15분)

[컨테이너]

```
cd /workspace/my-todo
python -c "
from todo import add_task, complete_task, delete_task, save_tasks, load_tasks
tasks = []
```

```
add_task(tasks, '우유 사기')
add_task(tasks, '운동하기')
complete_task(tasks, 1)
print(tasks)
delete_task(tasks, 2)
print(tasks)
save_tasks(tasks, 'tasks.json')
print(load_tasks('tasks.json'))
"
```

확인 기준:

- 첫 출력에서 1번 할 일의 `done` 이 `True` 여야 합니다.
- 둘째 출력에서 2번 할 일이 사라져야 합니다.
- 셋째 출력이 둘째 출력과 같아야 합니다.

[컨테이너]

```
rm -f tasks.json
```

7단계. 기록 쓰고 저장하기 (25분)

무엇을: `기록.md` 를 만들고 아래 질문에 한 문장씩 답합니다.

[컨테이너]

```
cd /workspace/my-todo
touch 기록.md
```

[Windows] VS Code 에서 `C:\harness-lab\my-todo\기록.md` 를 열고 아래 틀을 붙여 넣습니다.

```
# 6장 기록

## 1. AI 에게 시킨 일 (TASK_todo.md 의 목표 칸을 그대로 옮긴다)
답:

## 2. 계획에서 고치게 한 것 (검토 네 질문 중 아니오였던 번호. 없으면 "없음")
답:

## 3. 검증에서 실패한 것과 고친 방법 (없으면 "없음")
답:

## 4. 내가 이해한 것 세 문장
1. (add_task 가 할 일을 어떻게 만드는지)
2. (complete_task 가 무엇을 바꾸는지)
3. (save_tasks 와 load_tasks 가 무엇을 맡는지)

## 5. 테스트가 통과해도 확인해야 하는 것 한 가지
답:
```

확인 기준: 모든 `답:` 뒤에 글자가 있어야 합니다. 확인:

[컨테이너]

```
grep -c "^답:\s*$" 기록.md
```

결과가 `0` 이면 저장합니다. 저장은 사람이 합니다.

[컨테이너]

```
cd /workspace/my-todo
python verify.py
git add -A
```

```
git status
git commit -m "6장: 할 일 목록 완성 (R1~R4, 검증 통과, 기록 작성)"
git log --oneline
```

확인 기준: `git status` 에 `tasks.json` 이 없어야 합니다. 커밋 과정에서 `[pre-commit]` 이 "검증 통과"를 출력해야 합니다.

확인 체크리스트

- ■ 1단계 파일 여덟 개가 모두 있다.
- ■ `AGENTS.md` 에 `calc` 가 0건이다.
- ■ 3단계 표의 네 줄이 모두 "예"다.
- ■ 계획 검토 네 질문에 모두 답했다.
- ■ `python verify.py` 가 "검증 통과"다.
- ■ 테스트 파일이 원본과 같다.
- ■ 6단계 직접 실행의 세 출력을 확인했다.
- ■ `기록.md` 의 `답:` 칸이 모두 채워졌다.
- ■ 커밋이 한 줄 생겼다.

막혔을 때

증상	해결
`cp` 가 "No such file" 이라고 한다	파일 경로를 `ls /tutorial/templates` 로 확인한다.
테스트가 `ImportError` 를 낸다	todo.py 에 함수 이름이 빠졌다. 3단계 표의 함수 이름을 AI 에게 알리고 추가하게 한다.
AI 가 테스트를 고치려 한다	권한 설정이 막는다. 막히지 않았다면 `diff` 로 확인하고 원래대로 되돌린다.
커밋이 `[pre-commit]` 에서 취소된다	실패 줄을 읽고 5단계의 AI 문장으로 원인부터 설명하게 한다.

정답 코드는 제공하지 않습니다. 테스트 실패 줄을 하나씩 읽고, AI 와 대화하며 맞춰 갑니다.

다음 단계

이것으로 1단계가 끝납니다. 2단계에서는 같은 흐름을 **파일에 저장하는 프로그램, 상태 규칙, 요구사항 변경, 되돌리기**로 넓혀 갑니다.

부록. 템플릿과 예제

수업에서 복사해 쓰는 파일을 그대로 실었다. 주석을 따라 읽는다.

A-1. 요구사항 빈칸 틀 (2장)

```
# 요구사항 (빈칸을 채우는 틀)

> 2교시에 쓰는 틀이다. 무엇을 만들지 "이렇게 부르면 → 이런 결과" 형식으로 적는다.
> "어떻게 만들지"는 적지 않는다. 그것은 AI 가 정한다.

## 한 문장 요약
(무엇을 하는 프로그램인지 한 문장으로)

## 기능 요구사항

### R1. (기능 이름)
- 이렇게 하면: (입력 또는 사용 방법)
- 이런 결과가 나온다: (결과)
- 잘못 쓰면: (잘못된 입력일 때 무엇이 나오는가)

### R2. (기능 이름)
- 이렇게 하면:
- 이런 결과가 나온다:
- 잘못 쓰면:

## 만들지 않는 것
- (이번에 만들지 않을 기능을 두세 개)

---

## 체크리스트 (다 쓴 뒤 스스로 확인)
- [ ] 두 사람이 읽으면 같은 것을 만들 수 있는가?
- [ ] 모든 기능에 "잘못 쓰면" 항목이 있는가?
- [ ] 결과가 "잘 된다"처럼 흐릿하지 않고, 눈으로 확인할 수 있는가?
- [ ] "만들지 않는 것"이 적혀 있는가?
```

A-2. 작업 지시서 빈칸 틀 (3장)

```
# 작업 지시서 (빈칸을 채우는 틀)

> 3교시에 쓰는 틀이다. AI 에게 일을 시키기 전에, 아래 여섯 칸을 채워서 건넨다.
> 한 줄로 "프로그램 만들어 줘"라고 시키지 않는다.

## 1. 목표
(이 작업이 끝나면 무엇이 되어 있어야 하는가. 한두 문장)

## 2. 요구사항
(요구사항 문서의 번호. 예: R1, R2)

## 3. 범위
- 고쳐도 되는 파일:
- 고치면 안 되는 파일:

## 4. 하지 말 것
- (AI 가 덧붙이기 쉬운 것을 적는다. 예: 파일 저장 기능은 만들지 않는다)
- (새로 설치할 패키지가 있다면 적는다. 없으면 "새 패키지를 설치하지 않는다")

## 5. 완료 조건
- [ ] `python verify.py` 가 "검증 통과"로 끝난다.
- [ ] (눈으로 확인할 수 있는 조건을 한 줄씩)

## 6. 진행 방법
먼저 계획만 말한다. 내가 "진행"이라고 하기 전에는 파일을 고치지 않는다.
끝나면 무엇을 바꿨는지, 어떤 명령으로 확인했는지 보고한다.

---

## 좋은 지시와 나쁜 지시 비교
| 나쁜 지시 | 좋은 지시 |
|---|---|
| "할 일 앱 만들어 줘" | "R1~R3 을 구현한다. 파일 저장은 하지 않는다(메모리에만)." |
| "에러 고쳐 줘" | "`python verify.py` 결과를 붙인다. 원인을 먼저 설명하고 고쳐 줘." |
| "알아서 잘 해 줘" | "두 방법이 있으면 장단점을 말하고 내 선택을 기다려 줘." |
| "테스트 통과시켜 줘" | "todo.py 를 고쳐서 통과시켜 줘. tests/ 는 고치지 않는다." |
```

A-3. 규칙 문서 빈칸 틀 (4장)

```
# AGENTS.md — 작업 규칙 (빈칸을 채우는 틀)

> 4교시에 채운다. AI 에이전트(OpenCode)는 이 프로젝트에서 일을 시작할 때 이 파일을 먼저 읽는다.
> 규칙은 "부탁"이다. 잠금장치(opencode.jsonc)와 함께 있어야 힘이 생긴다.
> 빈칸 `[작성]` 을 채우고, 아래 "꼭 들어가야 할 것" 목록을 모두 체크한다.

## 1. 이 프로젝트는 무엇인가
- 목표: [작성]
- 사용자: [작성] (이 코드를 읽을 사람. 예: 프로그래밍을 처음 배우는 사람)

## 2. 파일 나누기
| 파일 | 고쳐도 되는가 |
|---|---|
| todo.py | [작성] |
| tests/ | [작성] |
| AGENTS.md, opencode.jsonc, verify.py | [작성] |

## 3. 일하는 순서
1. [작성] (먼저 무엇을 하는가)
2. [작성] (계획을 말하고 무엇을 기다리는가)
3. [작성] (구현)
4. [작성] (검증 방법)
5. [작성] (보고 방법)

## 4. 하지 말 것
- [작성] (커밋과 푸시는 누가 하는가)
- [작성] (테스트에 대해 무엇을 금지하는가)
- [작성] (새 패키지나 인터넷 사용에 대해)

## 5. 코드 규칙
- [작성] (주석은 어떤 언어로, 어디에 달 것인가)
- [작성] (이름 짓기, 한 줄에 한 가지 일 등)

## 6. 검증
- 완료라고 말하기 전에 [작성] 명령을 실행한다.
- 검증이 실패하면 [작성] (몇 번까지 고쳐 보고, 그다음에는 무엇을 하는가)

## 7. 보고 형식
```

[한 일] [바뀐 파일] [검증 결과] (python verify.py 의 마지막 줄) [확인 방법] [남은 일]

```
---

## 꼭 들어가야 할 것 (체크리스트)
- [ ] 계획을 먼저 말하고, 사람의 "진행"을 기다린다는 문장
- [ ] 테스트 파일을 고치지 않는다는 문장
- [ ] 커밋과 푸시는 하지 않는다는 문장
- [ ] `python verify.py` 로 검증한다는 문장
- [ ] 바뀐 파일과 확인 방법을 보고한다는 문장
```

A-4. 권한 설정 완성본 (4장)

```
// ============================================================
// opencode.jsonc - AI 에이전트(OpenCode) 권한 설정  [완성본]
//
// 4교시에 이 파일을 프로젝트 맨 위 폴더로 복사한다.
// 규칙(AGENTS.md)은 "부탁"이고, 이 파일은 "잠금장치"다.
//
// 읽는 법
//   action   : 무엇을 하려는가  (shell = 명령 실행, edit = 파일 고치기, read = 파일 읽기)
//   resource : 무엇에 대해서인가 (* 는 "아무 글자나")
//   effect   : allow(그냥 허용) / ask(나에게 물어봄) / deny(금지)
//
// 위에서 아래로 읽으며, 여러 규칙에 걸리면 "가장 아래 규칙"이 이긴다.
// 그래서 넓은 규칙을 위에, 예외를 아래에 쓴다.
// ============================================================
{
  "$schema": "https://opencode.ai/config.json",
  "model": "openrouter/anthropic/claude-haiku-5.5",
  "share": "disabled",

  "permissions": [
    // ---------- 1. 명령 실행 ----------
    // 기본: 모든 명령은 실행 전에 나에게 묻는다.
    { "action": "shell", "resource": "*", "effect": "ask" },
    // 검증과 읽기 명령은 묻지 않는다.
    { "action": "shell", "resource": "python verify.py", "effect": "allow" },
    { "action": "shell", "resource": "python -m pytest *", "effect": "allow" },
    { "action": "shell", "resource": "git status *", "effect": "allow" },
    { "action": "shell", "resource": "git diff *", "effect": "allow" },
    { "action": "shell", "resource": "git log *", "effect": "allow" },
    // 저장과 올리기는 사람만 한다. 에이전트는 금지.
    { "action": "shell", "resource": "git commit *", "effect": "deny" },
    { "action": "shell", "resource": "git push *", "effect": "deny" },
    { "action": "shell", "resource": "git reset *", "effect": "deny" },

    // ---------- 2. 파일 고치기 ----------
    // 규칙, 권한, 검증 스크립트, 테스트 기준은 에이전트가 고치지 못한다.
    { "action": "edit", "resource": "*AGENTS.md", "effect": "deny" },
    { "action": "edit", "resource": "*opencode.jsonc", "effect": "deny" },
    { "action": "edit", "resource": "*verify.py", "effect": "deny" },
    { "action": "edit", "resource": "*/tests/*", "effect": "deny" },
    { "action": "edit", "resource": "*pre-commit", "effect": "deny" },

    // ---------- 3. 비밀 값 ----------
    { "action": "read", "resource": "*.env", "effect": "deny" },

    // ---------- 4. 인터넷 ----------
    { "action": "webfetch", "resource": "*", "effect": "ask" },
    { "action": "websearch", "resource": "*", "effect": "ask" }
  ]
}
```

A-5. 검증 스크립트 (5장)

```
"""
verify.py - 검증 스크립트 (하네스의 "검증" 담당)

사용법 (프로젝트 폴더에서):  python verify.py
  - 테스트가 모두 통과하면 "검증 통과"를 출력하고 끝난다.
  - 하나라도 실패하면 "검증 실패"를 출력하고, 종료 코드 1 로 끝난다.

커밋 직전에 이 파일이 자동으로 실행된다(.githooks/pre-commit).
실패하면 커밋이 취소된다. 그래서 "테스트를 통과한 것만 저장된다"는 규칙이 사람의 기억이 아니라 도구로 지켜진다.
"""

import subprocess
import sys

# 파이썬으로 pytest 를 실행한다. 결과(통과/실패)는 pytest 가 정한다.
result = subprocess.run([sys.executable, "-m", "pytest", "-q"])

if result.returncode != 0:
    print("")
    print("검증 실패: 위의 오류를 고친 뒤 다시 실행하세요.")
    sys.exit(1)  # 0 이 아닌 값으로 끝나야 커밋이 취소된다

print("")
print("검증 통과: 이제 저장(commit)해도 됩니다.")
```

A-6. 커밋 전 검증 훅 (5장)

```
#!/bin/sh
# ============================================================
# pre-commit - 커밋 직전에 git 이 자동으로 실행하는 스크립트
#
# verify.py 가 실패하면(종료 코드 1) 이 스크립트도 1 로 끝나고,
# git 은 커밋을 만들지 않는다.
#
# 켜는 법 (프로젝트 폴더에서 한 번만):
#   git config core.hooksPath .githooks
# ============================================================

echo "[pre-commit] 저장하기 전에 검증을 실행합니다..."

python verify.py
result=$?

if [ $result -ne 0 ]; then
    echo ""
    echo "[pre-commit] 검증 실패: 커밋을 취소했습니다."
    exit 1
fi
```

A-7. 할 일 목록 요구사항 (6장)

```
# 요구사항: 할 일 목록 (6교시 미니 프로젝트)

## 한 문장 요약
할 일을 적고, 목록으로 보고, 끝낸 일을 표시하고, 지울 수 있는 명령줄 프로그램. 목록은 파일에 저장되어 프로그램을 껐다 켜도 남는다.

## 할 일 한 건의 모양
| 항목 | 설명 |
|---|---|
| `id` | 1, 2, 3 … 순서대로 붙는 번호. 지워도 다시 쓰지 않는다. |
| `title` | 할 일의 내용. 공백만 있으면 안 된다. |
| `done` | 끝났으면 `True`, 아직이면 `False`. 처음에는 `False`. |

## 기능 요구사항

### R1. 할 일 추가
- `add_task(tasks, title)` 을 부르면 새 할 일을 목록 끝에 넣고, 그 할 일을 돌려준다.
- 제목이 공백뿐이면 `ValueError` 를 일으킨다.

### R2. 완료 표시
- `complete_task(tasks, task_id)` 을 부르면 그 번호의 할 일을 `done = True` 로 바꾼다.
- 그 번호가 없으면 `ValueError` 를 일으킨다.

### R3. 삭제
- `delete_task(tasks, task_id)` 을 부르면 그 번호의 할 일을 목록에서 뺀다.
- 그 번호가 없으면 `ValueError` 를 일으킨다.

### R4. 파일 저장과 읽기
- `save_tasks(tasks, path)` 은 목록을 JSON 파일로 저장한다.
- `load_tasks(path)` 은 JSON 파일을 읽어 목록을 돌려준다.
- 파일이 아직 없으면 빈 목록 `[]` 을 돌려준다. 오류를 내지 않는다.

### R5. 명령줄에서 쓰기 (선택 사항)
- `python todo.py` 뒤에 `add`, `list`, `done`, `delete` 를 붙여 쓸 수 있다.
- 목록 보기는 번호, 완료 여부(`[x]` 또는 `[ ]`), 제목을 한 줄씩 보여 준다.

## 만들지 않는 것
- 마감일, 우선순위, 검색, 정렬
- 웹 화면, 데이터베이스, 여러 사람이 함께 쓰는 기능
```

A-8. 할 일 목록 작업 지시서 (6장)

```
# 작업 지시서: 할 일 목록 만들기 (6교시)

## 1. 목표
할 일을 추가하고, 완료 표시하고, 지우고, 파일에 저장할 수 있는 `todo.py` 를 만든다.

## 2. 요구사항
`REQUIREMENTS.md` 의 R1~R5 를 구현한다.

## 3. 범위
- 새로 만드는 파일: `todo.py`
- 고쳐도 되는 파일: `todo.py` 하나뿐이다.
- 고치면 안 되는 파일: `tests/` 폴더 전체, `verify.py`, `AGENTS.md`, `opencode.jsonc`

## 4. 하지 말 것
- 웹 화면, 데이터베이스, 외부 패키지(설치가 필요한 것)를 쓰지 않는다. 파이썬 기본 기능만 쓴다.
- 요구사항에 없는 기능(검색, 정렬, 마감일)을 덧붙이지 않는다.
- 테스트 파일의 기대값을 바꾸거나 테스트를 건너뛰게 만들지 않는다.

## 5. 완료 조건
- [ ] `python verify.py` 가 "검증 통과"로 끝난다.
- [ ] `tests/test_todo.py` 의 테스트가 모두 통과한다. (테스트 개수는 줄지 않았다)
- [ ] `todo.py` 의 모든 함수에 한국어 주석이 있다.
- [ ] `git diff tests/` 에 아무 변화가 없다.

## 6. 진행 방법
`AGENTS.md` 의 작업 절차를 따른다. 먼저 계획을 말하고, 내가 "진행"이라고 하면 구현한다.
끝나면 `AGENTS.md` 의 보고 형식으로 보고한다.
```

A-9. 할 일 목록 테스트 (6장)

```
"""
test_todo.py - 할 일 목록 프로그램(todo.py)이 지켜야 할 약속

6교시에 AI 에이전트는 이 테스트를 통과하도록 todo.py 를 만든다.
이 파일은 사람이 쓴 "기준"이다. AI 는 이 파일을 고칠 수 없다(opencode.jsonc 에서 막는다).

이 파일이 보여 주는 개념
  1) 기준(테스트)은 사람이 정하고, 구현(todo.py)은 AI 가 맡는다.
  2) 정상 경우와 잘못된 경우를 모두 약속한다.
  3) 파일 저장은 임시 폴더(tmp_path)에서만 한다. 내 진짜 파일을 건드리지 않는다.
"""

import pytest

from todo import add_task, complete_task, delete_task, load_tasks, save_tasks


def test_add_task_creates_new_task():
    tasks = []
    task = add_task(tasks, "우유 사기")
    assert task == {"id": 1, "title": "우유 사기", "done": False}
    assert tasks == [task]  # 목록에도 들어가야 한다


def test_add_task_numbers_go_up():
    tasks = []
    add_task(tasks, "첫째")
    second = add_task(tasks, "둘째")
    assert second["id"] == 2


def test_add_task_rejects_blank_title():
    # 공백만 있는 제목은 받지 않는다.
    with pytest.raises(ValueError):
        add_task([], "   ")


def test_complete_task_marks_done():
    tasks = []
    add_task(tasks, "운동하기")
    complete_task(tasks, 1)
    assert tasks[0]["done"] is True


def test_complete_task_not_found():
    with pytest.raises(ValueError):
        complete_task([], 99)


def test_delete_task_removes_it():
    tasks = []
    add_task(tasks, "지울 일")
    delete_task(tasks, 1)
    assert tasks == []


def test_delete_task_not_found():
    with pytest.raises(ValueError):
        delete_task([], 99)


def test_save_and_load_roundtrip(tmp_path):
```

```
    path = tmp_path / "tasks.json"  # 임시 폴더 안의 파일만 쓴다
    tasks = []
    add_task(tasks, "저장할 일")
    save_tasks(tasks, path)
    assert load_tasks(path) == tasks


def test_load_missing_file_returns_empty_list(tmp_path):
    # 파일이 아직 없으면 빈 목록을 돌려준다(오류가 나지 않는다).
    assert load_tasks(tmp_path / "없는파일.json") == []
```

A-10. 계산기 예제 (4·5장)

```
"""
calc.py - 아주 작은 계산 함수 모음 (1단계 연습용)

이 파일이 보여 주는 개념
  1) 함수는 "입력 → 계산 → 결과"에 이름을 붙인 것이다.
  2) 잘못된 입력(0으로 나누기)은 조용히 틀린 답을 내지 않고 오류로 알린다.

5교시 연습: multiply(곱하기) 함수가 아직 없다. 테스트를 먼저 실행하면 실패한다(빨강).
          AI 에게 multiply 를 추가하게 해서 테스트를 통과시킨다(초록).
"""


def add(a: float, b: float) -> float:
    """두 수를 더한다."""
    return a + b


def subtract(a: float, b: float) -> float:
    """a 에서 b 를 뺀다."""
    return a - b


def divide(a: float, b: float) -> float:
    """a 를 b 로 나눈다. b 가 0 이면 ValueError 오류를 일으킨다."""
    # 0 으로 나누면 계산할 수 없으므로 미리 막는다. 이 검사가 없으면 프로그램이 멈춘다.
    if b == 0:
        raise ValueError("0으로 나눌 수 없습니다.")
    return a / b
```

A-11. 계산기 테스트 (5장)

```
"""
test_calc.py - calc.py 가 약속대로 동작하는지 확인하는 테스트

이 파일이 보여 주는 개념
  1) 테스트는 "이 값을 넣으면 이 값이 나와야 한다"를 코드로 적은 약속이다.
  2) assert 뒤의 식이 참이면 통과, 거짓이면 실패다.
  3) 정상 경우뿐 아니라 잘못된 경우(0으로 나누기)도 약속에 넣는다.
"""

import pytest

from calc import add, divide, multiply, subtract


def test_add():
    assert add(2, 3) == 5


def test_subtract():
    assert subtract(10, 4) == 6


def test_divide():
    assert divide(9, 3) == 3


def test_divide_by_zero_raises_error():
    # 0 으로 나누면 ValueError 가 나와야 한다.
    with pytest.raises(ValueError):
        divide(1, 0)


def test_multiply():
    # 5교시에 AI 가 multiply 를 추가하면 이 테스트가 통과한다.
    assert multiply(4, 5) == 20
```